

从 LangChain 设计思想到 Java Spring Boot 实现

参考 [LangChain 官方文档](#) 和 [OpenAI Cookbook](#) 的设计思想，用 Java Spring Boot 重新实现。

写这篇文档的目的是：**通过对照 LangChain 的设计，理解我们 Java 项目里每个组件在做什么。**

一、LangChain 的核心设计理念

1.1 一句话总结

`Agent = Model + Tools + System Prompt`

LangChain 把一个 Agent 拆成三个东西： - **Model**（模型）：调哪个 LLM（如 Claude）
- **Tools**（工具）：LLM 可以调用哪些函数 - **System Prompt**（系统提示词）：告诉 LLM 它是什么角色、有哪些工具

1.2 内部循环（ReAct 模式）

LangChain 的 Agent 内部这样运行：

用户问题

- **Thought**（LLM 思考该怎么做）
- **Action**（LLM 决定调用哪个工具）
- **Action Input**（LLM 决定传什么参数）
- **Observation**（程序执行工具，得到结果）
- **Thought**（LLM 看结果后再思考）
- 如果继续调工具 → 回到 **Action**
- 如果答案够了 → **Final Answer**（返回给用户）

用伪代码表示就是：

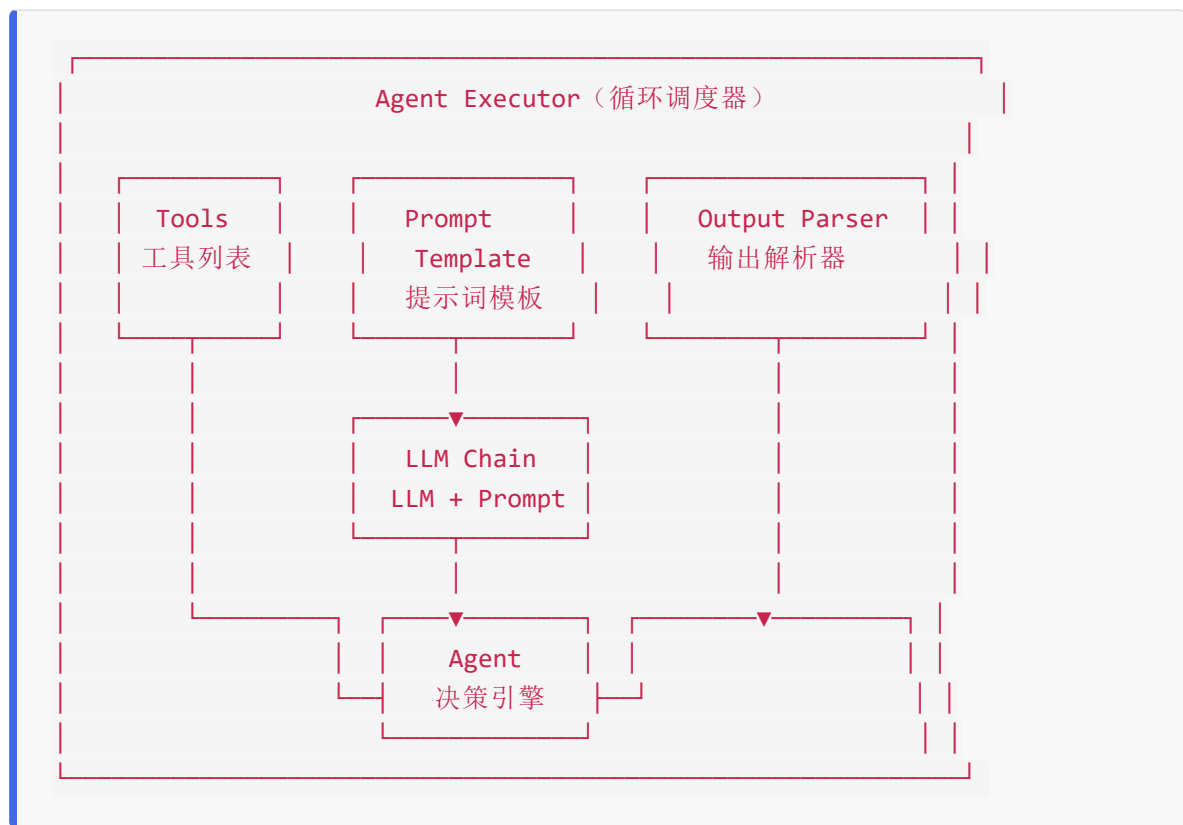
```
while (true) {  
    问 LLM: "你该怎么做?"  
    LLM 回答: "我要调用 file_read 工具, 参数是 path=main.java"  
  
    if (LLM 说要调工具) {  
        执行工具 file_read("main.java")  
        把结果告诉 LLM: "文件内容是 XXX"  
        继续循环  
    } else {  
        LLM 已经给出了最终答案  
        返回给用户  
    }  
}
```

二、LangChain 的六个组件

OpenAI Cookbook 里的 LangChain 实现用了六个组件:

#	组件	它做什么	在我们的 Java 项目中对应什么
1	Tool	一个可以被 LLM 调用的函数, 带名字和描述	AgentTool 接口 + FileReadTool / FileWriteTool / BashTool
2	Prompt Template	组装提示词 (告诉 LLM 它有什么工具、怎么做)	在 AnthropicClient 里手动构建请求体
3	Output Parser	把 LLM 的文本回复解析成结构化的"要调哪个工具、传什么参数"或"最终答案"	AnthropicClient 里解析 JSON 响应中的 tool_use
4	LLM Chain	LLM + Prompt, 输入提示词, 输出 LLM 的回复	AnthropicClient.chat() 方法
5	Agent	把 LLM Chain + Output Parser + Tools 组装成一个决策引擎	AgentService 类 (核心循环逻辑)
6	Agent Executor	跑"思考 → 行动 → 观察"循环, 直到得到最终答案	AgentService.run() 里的 for 循环

用图表示：



三、每个组件的详细解释

3.1 Tool (工具)

LangChain 里怎么写：

```
from langchain.tools import tool

@tool
def get_weather(city: str) -> str:
    """Get weather for a given city."""
    return f"It's always sunny in {city}!"
```

我们 Java 里怎么写：

```
// 1. 定义接口（契约）
public interface AgentTool {
    String getName();           // 工具名
    String getDescription();    // 工具描述（告诉 LLM 什么时候用）
    String execute(Map<String, Object> input); // 执行
}

// 2. 实现一个工具
@Component // 告诉 Spring: "自动创建这个类的实例"
public class FileReadTool implements AgentTool {

    @Override
    public String getName() { return "file_read"; }

    @Override
    public String getDescription() {
        return "读取指定路径的文件内容";
    }

    @Override
    public String execute(Map<String, Object> input) throws Exception {
        String path = (String) input.get("path");
        return Files.readString(Path.of(path));
    }
}
```

白话解释：

- Python 版用一个 `@tool` 注解 + 函数 docstring 来描述工具
- Java 版用一个接口 + 三个方法来做同样的事
- `getName()` 返回工具名（LLM 通过这个名字调用工具）
- `getDescription()` 返回工具描述（LLM 根据描述决定什么时候用它）
- `execute()` 是真正干活的代码

3.2 Prompt Template（提示词模板）

LangChain 里怎么写：

```
# Python 版：提示词控制 LLM 的行为
template = """
你是一个助手。你有以下工具：
{tools}

按这个格式回复：
Question: 用户的问题
Thought: 你的思考
Action: 要调用的工具名
Action Input: 传给工具的参数
Observation: 工具返回的结果
...（可以重复多轮）
Final Answer: 最终答案
"""
```

我们 Java 里怎么写：

在 `AnthropicClient.java` 的 `chat()` 方法里手动构建请求体：

```
// 构建发给 Claude API 的 JSON
Map<String, Object> body = new HashMap<>();
body.put("model", "claude-sonnet-4-20250514");
body.put("max_tokens", 4096);
body.put("messages", history.stream()
    .map(m -> Map.of("role", m.role(), "content", m.content()))
    .toList());
// 注意：我们没有显式的 Prompt Template。
// Claude API 自己处理"工具调用"的格式，不需要我们写 Thought/Action 模板。
```

白话解释：

- LangChain (OpenAI API) 需要你写显式的提示词模板，告诉 LLM "用 Thought → Action → Observation 格式回复"
- Claude API 原生支持工具调用，不需要我们写这个格式——Claude 自己知道怎么返回 `tool_use`
- 所以我们的 Java 版可以省略这个组件，直接传消息列表 + 工具列表即可

3.3 Output Parser (输出解析器)

LangChain 里怎么写：

```
# Python 版：用正则从 LLM 的文本回复里提取结构化信息
class OutputParser:
    def parse(self, llm_output):
        if "Final Answer:" in llm_output:
            return "最终答案" # 停止循环
        else:
            # 用正则提取 Action 和 Action Input
            match = re.match(r"Action: (.*)\n.*Action Input: (.*)",
llm_output)
            return ("调用工具", match.group(1), match.group(2))
```

我们 Java 里怎么写：

在 `AnthropicClient.java` 里解析 Claude API 返回的 JSON：

```
// Claude API 返回的 JSON 格式（比 LangChain 的文本格式更结构化）：
// {
//   "content": [
//     {"type": "text", "text": "我来帮你读文件"},
//     {"type": "tool_use", "id": "call_1", "name": "file_read",
//       "input": {"path": "main.java"}}
//   ]
// }

// 我们的解析代码：
JsonNode contentArray = responseJson.get("content");

// 遍历 content 数组的每个元素
for (JsonNode element : contentArray) {
    String type = element.get("type").asText();

    if ("text".equals(type)) {
        text = element.get("text").asText(); // 记录文本
    }
    if ("tool_use".equals(type)) {
        // 提取工具名、参数，创建 ToolCall 对象
        toolCalls.add(new ToolCall(
            element.get("id").asText(),
            element.get("name").asText(),
            input
        ));
    }
}
```

白话解释：

- LangChain (OpenAI API) 的 LLM 返回的是**纯文本**，所以需要正则去解析"Action: xxx"这种格式
- Claude API 返回的是**结构化 JSON**，有 `type` 字段区分文本和工具调用，不需要正则解析
- 两种方式本质一样：把 LLM 的输出转换成程序能处理的数据结构

3.4 LLM Chain (LLM + Prompt 的组合)

LangChain 里怎么写：

```
# Python 版: LLM + Prompt
llm = ChatOpenAI(model="gpt-4")
llm_chain = LLMChain(llm=llm, prompt=prompt_template)
response = llm_chain.run(input="加拿大有多少人口? ")
```

我们 Java 里怎么写：

```
// Java 版: AnthropicClient 把构建请求 + 发 HTTP + 解析响应放在一个方法里
public AnthropicResponse chat(List<Message> history) throws Exception {
    // 1. 构建请求体 JSON
    Map<String, Object> requestBody = buildRequestBody(history);

    // 2. 发 HTTP POST
    HttpRequest request = HttpRequest.newBuilder()
        .uri(URI.create(properties.apiEndpoint()))
        .header("x-api-key", properties.apiKey())
        .POST(HttpRequest.BodyPublishers.ofString(toJson(requestBody)))
        .build();
    HttpResponse<String> response = httpClient.send(request, ...);

    // 3. 解析响应 JSON
    return parseResponse(response.body());
}
```

白话解释：

- 我们简化了——不需要 LLMChain 这个抽象层
- 直接把构建请求、发 HTTP、解析响应放在一个 `chat()` 方法里
- 方法输入是消息列表，输出是 Claude 的响应（文本 + 工具调用）

3.5 Agent (决策引擎)

LangChain 里怎么写:

```
# Python 版: 把 LLM Chain + Output Parser + Tools 组装起来
agent = LLMSingleActionAgent(
    llm_chain=llm_chain,          # LLM + Prompt
    output_parser=output_parser, # 怎么解析 LLM 的输出
    stop=["\nObservation:"],    # 什么时候停止
    allowed_tools=["Search", "Calculator"] # 有哪些工具
)
```

我们 Java 里怎么写:

```

// Java 版: AgentService 是一个类, 包含所有需要的依赖 + 循环逻辑
@Service
public class AgentService {

    @Autowired private AnthropicClient anthropicClient; // LLM Chain
    @Autowired private ToolRegistry toolRegistry;        // Tools

    @Async
    public CompletableFuture<String> run(String userMessage) {
        List<Message> history = new ArrayList<>();
        history.add(new Message("user", userMessage));

        // 循环逻辑 (就是 Agent 的"决策"部分)
        for (int round = 0; round < 10; round++) {
            // 调 LLM
            AnthropicClient.AnthropicResponse response =
                anthropicClient.chat(history);

            // 如果没有工具调用 → 返回结果
            if (response.toolCalls().isEmpty()) {
                return CompletableFuture.completedFuture(response.text());
            }

            // 执行工具
            for (ToolCall tc : response.toolCalls()) {
                ToolResult result = toolRegistry.execute(tc.name(),
                    tc.input());
                history.add(new Message("user", "工具结果: " +
                    result.content()));
            }
            return CompletableFuture.completedFuture("超过最大轮数");
        }
    }
}

```

白话解释:

- LangChain 用一个专门的 Agent 对象做这件事
- 我们用 `AgentService` 类做同样的事——它的 `run()` 方法就是决策逻辑的核心
- `@Autowired` 注入的 `anthropicClient` 和 `toolRegistry` 就是 Agent 需要的 LLM Chain 和 Tools

3.6 Agent Executor (循环调度器)

LangChain 里怎么写：

```
# Python 版：Agent Executor 跑循环
agent_executor = AgentExecutor.from_agent_and_tools(
    agent=agent,      # 把 Agent 传进去
    tools=tools,      # 把工具列表传进去
    memory=memory     # 把记忆传进去（可选）
)

# 用户只需要调用这一行
result = agent_executor.run("加拿大有多少人口？")
```

我们 Java 里怎么写：

Agent Executor 的逻辑就在 `AgentService.run()` 里的 `for` 循环。

```
// LangChain 的 Agent Executor 本质上就做这件事：
for (int round = 0; round < 10; round++) {
    // 调 LLM → 解析输出 → 执行工具 → 循环
}
```

白话解释：

- LangChain 把循环单独抽成一个 Agent Executor 组件
- 我们把循环直接写在 `AgentService.run()` 方法里
- 本质一样——都是一个 while/for 循环

四、LangChain v1 (`create_agent`) 的简化

LangChain 官方在 v1 版本里把六个组件简化成了一个函数：

```
from langchain.agents import create_agent

agent = create_agent(
    model="claude-sonnet-4-6",          # 替代了 LLM Chain
    tools=[get_weather, search],        # 替代了显式的工具注册
    system_prompt="You are a helpful..." # 替代了 Prompt Template
)

result = agent.invoke(
    {"messages": [{"role": "user", "content": "What's the weather?"}]}
)
```

和我们的对比:

```
// 我们的 Java 版（等价于 create_agent() + agent.invoke()）:
@Autowired
private AgentService agentService;

// 用户只需要调用这一行
CompletableFuture<String> result = agentService.run("What's the
weather?");
```

LangChain v1	我们的 Java 版
model="claude-sonnet-4-6"	AgentProperties.model() 配置在 yml 里
tools=[get_weather, search]	AgentTool 接口 + @Component 自动注册
system_prompt="You are..."	(Claude API 原生支持, 不需要显式 prompt)
agent.invoke({"messages": [...]})	agentService.run(userMessage)
Output Parser (内部自动)	AnthropicClient 里手动解析 JSON
Agent Executor (内部循环)	AgentService.run() 里的 for 循环

五、从请求到响应的完整链路

下面跟踪用户问"帮我读 main.java"的完整过程，标注每一步对应的 LangChain 组件：

1. 用户发请求

```
POST /api/chat {"message": "帮我读 main.java"}
→ ChatController.chat()
→ agentService.run("帮我读 main.java")
[这一步对应 LangChain 的 agent.invoke()]
```

2. AgentService.run() 启动 [= Agent Executor 的循环开始]

```
→ history = [Message("user", "帮我读 main.java")]
```

3. 第 1 轮循环

```
→ anthropicClient.chat(history) [= LLM Chain]
→ 构建请求体、发 HTTP 到 Claude API
→ Claude 返回 JSON:
{"content": [
  {"type": "tool_use", "name": "file_read",
   "input": {"path": "main.java"}}
]}
→ 解析 JSON [= Output Parser]
→ 返回 AnthropicResponse(text="", toolCalls=[ToolCall("file_read",
{path: "main.java"})])
```

```
→ toolCalls 不为空 → 执行工具 [= Tool 执行]
→ toolRegistry.execute("file_read", {path: "main.java"})
→ 找到 FileReadTool
→ FileReadTool.execute({"path": "main.java"})
→ 返回文件内容 "public class Main { ... }"
→ 结果加入 history
```

4. 第 2 轮循环 [= Agent Executor 继续循环]

```
→ anthropicClient.chat(history) [本次 history 包含了文件内容]
→ Claude 返回 JSON:
{"content": [
  {"type": "text", "text": "文件内容如下: public class Main { ...
}"}
]}
→ toolCalls 为空 → 返回 AnthropicResponse(text="文件内容如下...",
toolCalls=[])
```

```
→ toolCalls 为空 → 循环结束
→ return "文件内容如下: public class Main { ... }"
```

5. ChatController 收到结果 → 返回给用户

六、我们 Java 版的简化之处

LangChain 做的事情	我们是怎么做的	为什么简化
Prompt Template (手写 Thought/Action 格式)	Claude API 原生支持 tool_use	Claude API 不需要这种格式
Output Parser (正则解析 LLM 文本)	解析 Claude API 的 JSON	JSON 比文本更容易解析
LLM Chain (LLM + Prompt 组合)	AnthropicClient.chat() 一个方法	不需要额外的抽象层
Agent (组装 LLM+Parser+Tools)	AgentService.run() 直接引用依赖	用 @Autowired 注入, 更简单
Agent Executor (循环调度器)	for 循环写在 run() 方法里	不需要单独的类

七、学到了哪些 Java/Spring Boot 知识

通过实现这个项目，你学到了：

知识点	在项目里的体现
接口	<code>AgentTool</code> 接口定义了工具的契约
实现接口	<code>FileReadTool implements AgentTool</code>
<code>@Component</code>	标记类让 Spring 自动创建实例
<code>@Service</code>	和 <code>@Component</code> 一样，语义上表示"业务逻辑类"
<code>@Autowired</code>	把 Spring 容器里的实例连接到这个字段
<code>@Async</code>	方法在独立线程执行
<code>@RestController</code>	标记 HTTP 控制器
<code>@PostMapping</code>	注册 HTTP POST 路由
<code>@RequestBody</code>	把 JSON 请求体自动转成 Java 对象
<code>@RequestParam</code>	从 URL 参数获取值
<code>@ConfigurationProperties</code>	把 yml 配置映射到 Java 对象
<code>Record</code>	不可变数据类
<code>List/Map</code>	集合框架
<code>Stream API</code>	链式数据处理
<code>HttpClient</code>	Java 内置的 HTTP 客户端
<code>Jackson</code>	JSON 和 Java 对象的互转
<code>SseEmitter</code>	服务器向客户端推送数据
<code>CompletableFuture</code>	异步任务
<code>@EnableAsync</code>	启用异步支持

八、C++ 开发者看 LangChain 思想的总结

LangChain 的设计思想说白了就是：

- 1. 定义一个 while 循环

2. 循环里做四件事：调 LLM → 解析 LLM 的回复 → 如果是调工具就执行工具 → 把结果发回 LLM → 继续循环
3. 直到 LLM 说"我回答完了"，退出循环，把答案给用户

这和你在 C++ 里写 RPC 服务很像：

```
// C++ RPC Server 的请求处理循环
while (running) {
    Request req = server.receive();    // 接收请求
    string method = parse_method(req); // 解析请求方法
    if (method == "get_weather") {
        string result = get_weather(req.params); // 执行对应方法
        server.send(result);                // 返回结果
    }
}
```

LangChain Agent 的循环也是这个结构：

```
while (true) {
    Response resp = llm.chat(history); // 调 LLM（类似收请求）
    if (resp.hasToolCall()) {
        String result = executeTool(); // 执行工具（类似执行 RPC 方法）
        history.add(result);           // 把结果发回 LLM（类似返回响应）
    } else {
        return resp.text();            // 最终答案（循环结束）
    }
}
```

区别只是：RPC Server 是等客户端发请求，Agent 是等 LLM 告诉它下一步做什么。

关于 LangChain 的命名：

- **LangChain** = 官方 Python 版 (docs.langchain.com)
- **LangChain4j** = 社区 Java 移植版（非官方，不建议依赖）
- 我们的做法：**理解了 LangChain 的设计思想，用纯 Java/Spring Boot 自己实现，不引入任何第三方 LLM 框架**